Student Supervisor System Plan

MERN Stack + Cloudinary Object Storage Integration

System Architecture, Data Models, API Map, and Functional Requirements

PREPARED BY

Mohamed Abdirahman Muse • **Xanaan Abdirahman Mohamud** • **Najma Abdikadir Muse** • **Mohamed Mahdi Abdikarim** • **Abdullahi Ahmed Jimcaale** • **Nasro Hassan Mohamed**

1. Overview

Student Supervisor is a web-based management platform designed to replace fragmented communication channels with a structured, step-by-step submission → review → feedback workflow across three distinct roles: **Admin**, **Supervisor**, and **Student**.

- **Core Stack:** MongoDB, Express.js, React (Vite), and Node.js (MERN).
- **Object Storage: Cloudinary** for asset hosting, secure document uploads (PDF, DOCX, images), transformed access, and metadata storage.
- **Scope Focus:** Submission lifecycle, milestone tracking, formal review outcomes, and automated email notifications.

2. System Roles

Role	Description & Scope
Admin	Manages user accounts, creates group/cohort rooms, assigns students to supervisors, monitors global submission activity, and configures global settings.
Supervisor	Publishes milestone guidelines and tasks, reviews student submissions, approves work or requests changes with detailed feedback, and tracks student progress.
Student	Views assigned supervisor details, reviews published guidelines, submits work per milestone, tracks feedback/statuses, and resubmits updated work versions.

3. Functional Requirements (FR)

3.1 Admin Module

	Requirement Specification
FR-A1	Admin can create, edit, deactivate, and delete Supervisor and Student accounts.
FR-A2	Admin can create "Rooms" / "Groups" (e.g., cohorts, batches, subject tracks) to organize students.
FR-A3	Admin can assign one or more students to a specific supervisor, within or across groups.
FR-A4	Admin can reassign a student to a different supervisor.
FR-A5	Admin can view a dashboard of all groups, supervisors, and their assigned students.
FR-A6	Admin can view overall submission and approval activity across the platform.
FR-A7	Admin can define global settings (e.g., academic terms, submission categories/milestones, chapter/task templates).

3.2 Supervisor Module

	Requirement Specification
FR-S1	Supervisor can view the list of students assigned to them.
FR-S2	Supervisor can create and publish guidelines/documentation, structured per milestone (e.g., Chapter 1, Chapter 2).
FR-S3	Supervisor can view all submissions made by their assigned students.
FR-S4	Supervisor can open a submission and review its content and Cloudinary-hosted attached files.
FR-S5	Supervisor can **Approve** a submission, marking the milestone as complete.

	Requirement Specification
FR-S6	Supervisor can **Request Changes**, attaching comments describing required corrections.
FR-S7	Supervisor can leave general or itemized feedback comments on a submission.
FR-S8	Supervisor can track the overall progress of each assigned student (milestones completed vs. pending).

3.3 Student Module

	Requirement Specification
FR-T1	Student can view their assigned supervisor's profile and contact information.
FR-T2	Student can view guidelines/documentation published by their supervisor, per milestone.
FR-T3	Student can submit work (files/text) against a specific milestone (e.g., "Chapter 2").
FR-T4	Student can resubmit work after receiving change requests, preserving version submission history.
FR-T5	Student can view the status of each submission (Pending / Approved / Changes Requested).
FR-T6	Student can view supervisor comments and feedback on each submission.
FR-T7	Student can view their own overall progress (milestones completed vs. remaining).
FR-T10	Student receives notifications when a guideline/task is published or feedback is given.

3.4 Shared / Cross-Cutting

	Requirement Specification
FR-C1	All users can log in securely with role-based access control (Admin, Supervisor, Student).

	Requirement Specification
FR-C2	All submissions and guideline documents support file attachments via Cloudinary SDK/API.
FR-C3	The system provides a searchable activity/audit log per student-supervisor relationship.
FR-C4	Users can update their profile and account settings.

4. Email Notification Requirements

Automated email notifications keep supervisors and students updated on project milestones without requiring continuous manual checking.

	Trigger Event	Recipient(s)	Notification Details
FR-N1	Admin assigns a student	Supervisor & Student	Both parties receive an email confirmation of assignment with links to relevant profiles.
FR-N2	Supervisor publishes guideline	Assigned Student(s)	Student receives an email stating new guidance/documentation is available.
FR-N3	Supervisor creates task	Assigned Student(s)	Student receives an email with task details, requirement specifications, and deadline.
FR-N4	Student submits work	Assigned Supervisor	Supervisor receives notification that a milestone submission is ready for evaluation.

	Trigger Event	Recipient(s)	Notification Details
FR-N5	Review decision made	Submitting Student	Student is notified of approval or requested changes along with reviewer comments.

5. Technology Infrastructure

Layer	Technology	Purpose & Scope
Frontend	React (Vite) + Tailwind CSS	Responsive Web Dashboards for Admin, Supervisor, and Student roles
Backend API	Node.js + Express.js	RESTful API, JWT authentication, authorization middleware, business logic
Database	MongoDB (Atlas) via Mongoose	Document store for users, groups, milestones, submissions, notifications, and logs
File Storage	**Cloudinary**	Cloud object storage, PDF/DOCX asset hosting, secure signed client direct uploads
Auth	JWT (Access + Refresh Tokens)	Role-based authentication and route guarding (FR-C1)
Email	Resend / SendGrid / AWS SES	Automated email notifications (FR-N1 to FR-N5)
Hosting	Vercel / Render / Railway	Frontend and Express API deployment

6. Database Design (MongoDB - 7 Collections)

6. Database Design (MongoDB - 6 Collections)

The schema comprises **6 core collections** designed to support functional requirements cleanly while preserving data integrity.

6.1 Collections Summary

- **users:** Stores accounts for Admins, Supervisors, and Students with role designations and relationship mappings.
- **groups:** Represents cohorts, rooms, or batches (FR-A2).
- **milestones:** Guideline documents and tasks published by supervisors, with embedded Cloudinary asset records.
- **submissions:** Student work per milestone, maintaining version history (`versions[]`) and Cloudinary asset references (FR-T4).
- **notifications:** Tracks in-app alerts and email dispatch statuses.
- **auditlogs:** Searchable activity log for administrative oversight (FR-C3).

6.2 Mongoose Schema Implementation

```
// models/User.js & Group.js & Attachment Schema
const mongoose = require('mongoose');
const { Schema } = mongoose;

const userSchema = new Schema({
  fullName: { type: String, required: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  passwordHash: { type: String, required: true },
  role: { type: String, enum: ['admin', 'supervisor', 'student'], required: true },
  isActive: { type: Boolean, default: true },
  supervisorId: { type: Schema.Types.ObjectId, ref: 'User', default: null },
  groupId: { type: Schema.Types.ObjectId, ref: 'Group', default: null }
}, { timestamps: true });

const groupSchema = new Schema({
  name: { type: String, required: true },
  description: String,
```

```
  createdBy: { type: Schema.Types.ObjectId, ref: 'User', required: true }
}, { timestamps: { createdAt: true, updatedAt: false } });

// Reusable Cloudinary Attachment Schema
const attachmentSchema = new Schema({
  originalFilename: { type: String, required: true },
  publicId: { type: String, required: true },
  secureUrl: { type: String, required: true },
  format: String,
  bytes: Number
}, { _id: false });
```

```
// models/Milestone.js & Submission.js
const milestoneSchema = new Schema({
  supervisorId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  groupId: { type: Schema.Types.ObjectId, ref: 'Group', default: null },
  title: { type: String, required: true },
  description: String,
  order: { type: Number, default: 1 },
  attachments: [attachmentSchema],
  dueDate: Date
}, { timestamps: true });

const versionSchema = new Schema({
  versionNumber: Number,
  files: [attachmentSchema],
  note: String,
  submittedAt: { type: Date, default: Date.now }
}, { _id: false });

const commentSchema = new Schema({
  authorId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  content: { type: String, required: true },
  createdAt: { type: Date, default: Date.now }
}, { _id: false });

const submissionSchema = new Schema({
  milestoneId: { type: Schema.Types.ObjectId, ref: 'Milestone', required: true },
  studentId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  status: {
    type: String,
    enum: ['pending', 'approved', 'changes_requested'],
    default: 'pending'
  },
  versions: [versionSchema],
  comments: [commentSchema],
```

```
  reviewedBy: { type: Schema.Types.ObjectId, ref: 'User', default: null },
  reviewedAt: Date
}, { timestamps: true });

submissionSchema.index({ milestoneId: 1, studentId: 1 }, { unique: true });
```

```
// models/Notification.js & AuditLog.js
const notificationSchema = new Schema({
  userId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  type: {
    type: String,
    enum: ['assignment', 'guideline_published', 'task_created', 'submission_received',
'review_outcome'],
    required: true
  },
  message: { type: String, required: true },
  link: String,
  emailSent: { type: Boolean, default: false },
  isRead: { type: Boolean, default: false }
}, { timestamps: { createdAt: true, updatedAt: false } });

const auditLogSchema = new Schema({
  userId: { type: Schema.Types.ObjectId, ref: 'User' },
  action: { type: String, required: true },
  entityType: { type: String, required: true },
  entityId: Schema.Types.ObjectId
}, { timestamps: { createdAt: true, updatedAt: false } });
```

7. File Storage — Cloudinary Integration

Integrating Cloudinary allows direct uploads from the client to Cloudinary's secure REST endpoints, avoiding server bottlenecking while retaining authorization control via signed signature tokens.

Upload Workflow Architecture:

1. **Signature Request:** Client sends an authenticated request to Express server (POST `/api/uploads/signature`) specifying target folder parameters and timestamp.
2. **Signature Generation:** Backend generates an HMAC SHA-256 signature using the Cloudinary API Secret.

3. **Direct Cloudinary Upload:** Client uploads file directly to Cloudinary REST API (`https://api.cloudinary.com/v1_1/{cloud_name}/auto/upload`).
4. **Document Persistence:** Cloudinary returns asset attributes (`public_id`, `secure_url`, `format`, `bytes`). Client submits this payload to Express API to update MongoDB records.

8. API Endpoint Map

Method	Endpoint Path	Role Access	Requirement Mapping
POST	/api/auth/login	All Roles	FR-C1
POST	/api/users	Admin	FR-A1
PATCH	/api/users/:id	Admin	FR-A1, FR-A4
DELETE	/api/users/:id	Admin	FR-A1
POST	/api/groups	Admin	FR-A2
GET	/api/admin/dashboard	Admin	FR-A5, FR-A6
PATCH	/api/settings	Admin	FR-A7
GET	/api/supervisors/:id/students	Supervisor	FR-S1
POST	/api/milestones	Supervisor	FR-S2
GET	/api/milestones/:id/submissions	Supervisor	FR-S3
GET	/api/submissions/:id	Supervisor, Student	FR-S4
PATCH	/api/submissions/:id/approve	Supervisor	FR-S5
PATCH	/api/submissions/:id/request-changes	Supervisor	FR-S6
POST	/api/submissions/:id/comments	Supervisor, Student	FR-S7, FR-T6

Method	Endpoint Path	Role Access	Requirement Mapping
GET	/api/students/:id/progress	Supervisor, Student	FR-S8, FR-T7
GET	/api/students/:id/supervisor	Student	FR-T1
GET	/api/students/:id/milestones	Student	FR-T2
POST	/api/submissions	Student	FR-T3, FR-T4
POST	/api/uploads/signature	Supervisor, Student	FR-C2, Section 7
GET	/api/notifications	All Roles	FR-T10, Section 4
GET	/api/audit-logs	Admin	FR-C3
PATCH	/api/me	All Roles	FR-C4

9. Core Workflow

1. **User Provisioning & Group Allocation:** Admin creates Supervisor and Student accounts, forms cohort groups, and links students to supervisors → **FR-N1** email triggered.
2. **Milestone Guidance Publishing:** Supervisor creates milestone instructions and attaches reference files stored on Cloudinary → **FR-N2 / FR-N3** emails triggered.
3. **Student Work Submission:** Student reads requirements, uploads completed assignment to Cloudinary, and saves submission record → **FR-N4** email triggered to supervisor.
4. **Supervisor Review & Outcome:** Supervisor inspects submission and attached files, provides structured feedback, and executes either **Approve** or **Request Changes** → **FR-N5** email triggered to student.
5. **Iteration & Version History:** If revisions are requested, student submits an updated iteration (incrementing version history) for re-evaluation.
6. **Administrative Oversight:** Admin monitors aggregate milestone completion and pending review tasks via system dashboard (FR-A5, FR-A6).